

Sistema de gestión de tienda online de electrónicos

Shomara Berenice Rivera Huerta

10 de julio de 2026

Índice

1. Introducción y objetivos	2
2. Análisis y diseño	2
2.1. Diagrama entidad-relación (ER)	2
2.2. Justificación de la normalización	2
3. Implementación de la base de datos	3
4. Consultas y procedimientos almacenados	7
4.1. Consultas SQL	7
4.2. Procedimientos Almacenados (Store Procedures)	8
5. Resultados de Pruebas y Análisis de Rendimiento	12
5.1. Resultados de Consultas con Datos de Prueba	12
5.2. Análisis de Rendimiento con Índices (EXPLAIN)	12
5.3. Pruebas de Restricciones y Reglas de Negocio	13
6. Conclusiones y lecciones aprendidas	13

Lista de códigos

1. Script de creación de esquema, restricciones e índices	4
2. Script de inserción de datos de prueba	5
3. Consulta 1: Listar productos disponibles por categoría, ordenados por precio.	7
4. Consulta 2: Mostrar clientes con pedidos pendientes y total de compras.	7
5. Consulta 3: Reporte de los 5 productos con mejor calificación promedio en reseñas.	7
6. Store Procedures para automatizar reglas de negocio y mantener integridad de datos	8

1. Introducción y objetivos

En el entorno comercial actual, las tiendas de productos electrónicos necesitan sistemas eficientes para gestionar su gran volumen de operaciones diarias, llevar el control manual del inventario, los clientes y las ventas suele generar errores como venta de productos sin stock o historiales de compras perdidos.

El objetivo de este proyecto es diseñar e implementar una base de datos relacional y normalizada para una tienda online; el sistema busca centralizar la información mediante entidades bien definidas y el uso de llaves foráneas (*Foreign Keys*) que mantengan la integridad referencial. Además, se busca automatizar las reglas de negocio, como el límite de pedidos pendientes y la validación de compras para las reseñas, utilizando procedimientos almacenados (*Store Procedures*) e índices para asegurar un rendimiento óptimo.

2. Análisis y diseño

Para evitar la redundancia de datos y proteger la integridad de la base de datos, se estructuró el modelo dividiendo la información en entidades conectadas lógicamente.

2.1. Diagrama entidad-relación (ER)

Se determinó una estructura compuesta por 6 entidades principales:

- **Categorías:** Organiza los productos.
- **Productos:** Almacena el stock y los precios.
- **Clientes:** Registra la información de contacto de los compradores.
- **Pedidos:** Gestiona el estado y fecha de las compras.
- **Detalles_pedido:** Entidad puente que rompe la relación de muchos a muchos entre pedidos y productos, detallando cantidades.
- **Resenas:** Guarda las opiniones de los usuarios sobre los productos.

2.2. Justificación de la normalización

Al analizar el diagrama Entidad-Relación diseñado, logré identificar que cumple con las formas normales basándonos en las reglas:

1. **Primera Forma Normal (1NF):** El diagrama muestra que todos los atributos de cada entidad son atómicos, o sea que no tiene atributos multivaluados, por ejemplo, en lugar de guardar una lista de productos dentro de un mismo pedido, se creó la tabla **detalles_pedido**. Cada entidad está diseñada para almacenar un único valor por campo, cumpliendo la 1NF.

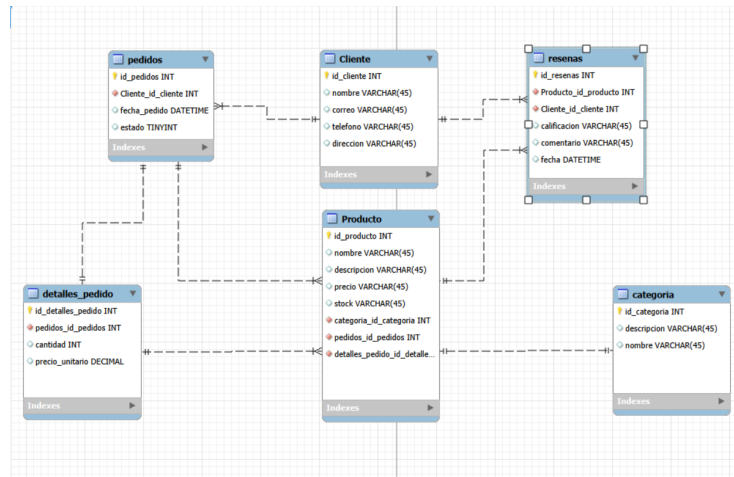


Figura 1: Diagrama entidad-relación del Sistema de tienda online

2. **Segunda Forma Normal (2NF):** El modelo cumple con la 2NF porque además de cumplir el requisito de satisfacer la 1FN, no tiene dependencias parciales. En las entidades que funcionan como puente (como `detalles_pedido`), los atributos dependen de la llave completa que identifica a ese detalle, y no solo de una parte de ella.
3. **Tercera Forma Normal (3NF):** Cumple con la 3NF ya que no existen dependencias transitivas; al observar las entidades, nos damos cuenta de que todos los atributos dependen solo de su llave principal, ningún atributo no clave depende de otro atributo no clave, en la tabla `productos` no ponemos el nombre de la categoría, solo su llave foránea.
4. **Cuarta Forma Normal (4NF):** Cumple con la 4NF porque no tenemos ninguna tabla que intente almacenar listas de datos independientes al mismo tiempo, quiere decir que no mezclamos cosas que no tienen nada que ver entre sí en un mismo lugar, separamos los pedidos de las reseñas para evitar que se generen combinaciones de datos sin sentido.
5. **Quinta Forma Normal (5NF):** Cumple con la 5NF porque las relaciones entre nuestras entidades son directas, paso a paso y de dos en dos (por ejemplo: el Cliente se relaciona con su Pedido, y el Pedido con el Producto a través del detalle), no tenemos relaciones donde 3 entidades distintas dependan estrictamente unas de otras al mismo tiempo.

3. Implementación de la base de datos

A continuación, se presentan los scripts para la creación de las tablas, la optimización con índices y el código de la inserción de datos de prueba.

Código 1: Script de creación de esquema, restricciones e índices

```

1
2 CREATE DATABASE IF NOT EXISTS tienda_electronica;
3 USE tienda_electronica;
4
5 -- Categorías
6 CREATE TABLE categorias (
7     id_categoria INT AUTO_INCREMENT PRIMARY KEY,
8     nombre VARCHAR(100) NOT NULL UNIQUE, -- Nombre único
9     descripcion TEXT
10 );
11
12 -- Clientes
13 CREATE TABLE clientes (
14     id_cliente INT AUTO_INCREMENT PRIMARY KEY,
15     nombre VARCHAR(150) NOT NULL,
16     correo VARCHAR(150) NOT NULL UNIQUE, -- Correo único
17     telefono VARCHAR(20),
18     direccion VARCHAR(255)
19 );
20
21 -- Productos
22 CREATE TABLE productos (
23     id_producto INT AUTO_INCREMENT PRIMARY KEY,
24     nombre VARCHAR(150) NOT NULL,
25     descripcion TEXT,
26     precio DECIMAL(10,2) NOT NULL,
27     stock INT NOT NULL CHECK (stock >= 0), -- Stock no negativo
28     id_categoria INT,
29     FOREIGN KEY (id_categoria) REFERENCES categorias(id_categoria)
30 );
31
32 -- Pedidos
33 CREATE TABLE pedidos (
34     id_pedido INT AUTO_INCREMENT PRIMARY KEY,
35     fecha_pedido DATE NOT NULL,
36     estado ENUM('pendiente', 'enviado', 'entregado') DEFAULT 'pendiente',
37     id_cliente INT,
38     FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente)
39 );
40
41 -- Detalles_pedido
42 CREATE TABLE detalles_pedido (
43     id_detalle INT AUTO_INCREMENT PRIMARY KEY,
44     id_pedido INT,
45     id_producto INT,
46     cantidad INT NOT NULL CHECK (cantidad > 0),
47     precio_unitario DECIMAL(10,2) NOT NULL,
48     FOREIGN KEY (id_pedido) REFERENCES pedidos(id_pedido),
49     FOREIGN KEY (id_producto) REFERENCES productos(id_producto)
50 );
51
52 -- Resenas
53 CREATE TABLE resenas (
54     id_resena INT AUTO_INCREMENT PRIMARY KEY,
55     calificacion INT NOT NULL CHECK (calificacion BETWEEN 1 AND 5),

```

```

56      comentario TEXT,
57      fecha DATE NOT NULL,
58      id_producto INT,
59      id_cliente INT,
60      FOREIGN KEY (id_producto) REFERENCES productos(id_producto),
61      FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente)
62 );
63
64 -- Optimizar la búsqueda de productos por nombre
65 CREATE INDEX idx_producto_nombre ON productos(nombre);
66
67 -- Optimizar la búsqueda de productos por su categoría
68 CREATE INDEX idx_producto_categoria ON productos(id_categoria);
69
70 -- Optimizar la búsqueda del historial de pedidos de un cliente
    específico
71 CREATE INDEX idx_pedido_cliente ON pedidos(id_cliente);

```

Código 2: Script de inserción de datos de prueba

```

1  USE tienda_electronica;
2
3  -- Categorías (3)
4  INSERT INTO categorias (nombre, descripcion) VALUES
5  ('Teléfonos', 'Smartphones y accesorios celulares'),
6  ('Laptops', 'Computadoras portátiles para uso personal y
    profesional'),
7  ('Accesorios', 'Cables, audífonos, fundas y periféricos');
8
9  -- Clientes (15)
10 INSERT INTO clientes (nombre, correo, telefono, direccion) VALUES
11 ('Pablo Hernandez', 'pablo@email.com', '5551234', 'Calle 1'),
12 ('Jhovanny Pérez', 'jhovanny@email.com', '5551235', 'Calle 2'),
13 ('Zury Bautista', 'zury@email.com', '5551236', 'Calle 3'),
14 ('Juan Ruiz', 'juan@email.com', '5551237', 'Calle 4'),
15 ('Angel Duenas', 'angel@email.com', '5551238', 'Calle 5'),
16 ('Hugo Soto', 'hugo@email.com', '5551239', 'Calle 6'),
17 ('Sara Rivera', 'sara@email.com', '5551240', 'Calle 7'),
18 ('Paco Luna', 'paco@email.com', '5551241', 'Calle 8'),
19 ('Cecilia Hernández', 'cecilia@email.com', '5551242', 'Calle 9'),
20 ('Raúl Mora', 'raul@email.com', '5551243', 'Calle 10'),
21 ('Nolan Rivera', 'nolan@email.com', '5551244', 'Calle 11'),
22 ('Omar Navarrete', 'omar@email.com', '5551245', 'Calle 12'),
23 ('Lupita Gonzalez', 'lupita@email.com', '5551246', 'Calle 13'),
24 ('José Ruz', 'jose@email.com', '5551247', 'Calle 14'),
25 ('Rogelio Navarrete', 'rogelio@email.com', '5551248', 'Calle 15');
26
27 -- Productos (30)(10 teléfonos, 10 laptops, 10 accesorios)
28 INSERT INTO productos (nombre, descripcion, precio, stock,
    id_categoria) VALUES
29 ('iPhone 13', 'Teléfono Apple', 15000, 20, 1), ('Galaxy S21', 'Telé
    fono Samsung', 12000, 15, 1),
30 ('Pixel 6', 'Teléfono Google', 10000, 10, 1), ('Xiaomi Mi 11', 'Tel
    éfono Xiaomi', 8000, 25, 1),
31 ('Moto G100', 'Teléfono Motorola', 7500, 30, 1), ('OnePlus 9', 'Tel
    éfono OnePlus', 11000, 5, 1),
32 ('iPhone 14', 'Teléfono Apple', 18000, 10, 1), ('Galaxy S22', 'Telé

```

```

fono Samsung', 14000, 12, 1),
33 ('Poco X3', 'Teléfono Poco', 5000, 40, 1), ('Huawei P40', 'Teléfono
    Huawei', 9000, 8, 1),
34 ('MacBook Air', 'Laptop Apple', 20000, 10, 2), ('Dell XPS 13', '
    Laptop Dell', 25000, 5, 2),
35 ('ThinkPad X1', 'Laptop Lenovo', 22000, 7, 2), ('HP Envy', 'Laptop
    HP', 18000, 12, 2),
36 ('Asus ZenBook', 'Laptop Asus', 17000, 9, 2), ('Acer Swift', '
    Laptop Acer', 15000, 15, 2),
37 ('MacBook Pro', 'Laptop Apple', 30000, 3, 2), ('Razer Blade', '
    Laptop Razer', 35000, 2, 2),
38 ('MSI Stealth', 'Laptop MSI', 28000, 4, 2), ('Lenovo Legion', '
    Laptop Gaming Lenovo', 24000, 6, 2),
39 ('AirPods Pro', 'Audífonos Apple', 5000, 50, 3), ('Galaxy Buds', '
    Audífonos Samsung', 3000, 40, 3),
40 ('Sony WH-1000XM4', 'Audífonos Sony', 6000, 15, 3), ('Cable USB-C',
    'Cable 2 metros', 200, 100, 3),
41 ('Cargador Rápido', 'Cargador 20W', 400, 80, 3), ('Funda iPhone 13'
    , 'Funda silicona', 300, 60, 3),
42 ('Mica de Cristal', 'Protector de pantalla', 150, 120, 3), ('
    PowerBank 10000mAh', 'Batería externa', 800, 30, 3),
43 ('Mouse Inalámbrico', 'Mouse Bluetooth', 500, 45, 3), ('Teclado Mec
    ánico', 'Teclado USB', 1200, 20, 3);
44
45 -- Pedidos (20)
46 INSERT INTO pedidos (fecha_pedido, estado, id_cliente) VALUES
47 ('2023-10-01', 'entregado', 1), ('2023-10-02', 'entregado', 2), ('
    2023-10-03', 'entregado', 3),
48 ('2023-10-04', 'enviado', 4), ('2023-10-05', 'pendiente', 5), ('
    2023-10-06', 'entregado', 6),
49 ('2023-10-07', 'enviado', 7), ('2023-10-08', 'pendiente', 8), ('
    2023-10-09', 'entregado', 9),
50 ('2023-10-10', 'enviado', 10), ('2023-10-11', 'pendiente', 11), ('
    2023-10-12', 'entregado', 12),
51 ('2023-10-13', 'enviado', 13), ('2023-10-14', 'pendiente', 14), ('
    2023-10-15', 'entregado', 15),
52 ('2023-10-16', 'enviado', 1), ('2023-10-17', 'pendiente', 2), ('
    2023-10-18', 'entregado', 3),
53 ('2023-10-19', 'enviado', 4), ('2023-10-20', 'pendiente', 5);
54
55 -- Detalles de Pedido (25)
56 INSERT INTO detalles_pedido (id_pedido, id_producto, cantidad,
    precio_unitario) VALUES
57 (1, 1, 1, 15000), (1, 21, 1, 5000), (2, 11, 1, 20000), (3, 2, 1,
    12000), (3, 22, 1, 3000),
58 (4, 12, 1, 25000), (5, 3, 2, 10000), (6, 23, 1, 6000), (7, 13, 1,
    22000), (8, 4, 1, 8000),
59 (9, 24, 3, 200), (10, 14, 1, 18000), (11, 5, 1, 7500), (12, 25, 2,
    400), (13, 15, 1, 17000),
60 (14, 6, 1, 11000), (15, 26, 1, 300), (16, 16, 1, 15000), (17, 7, 1,
    18000), (18, 27, 2, 150),
61 (19, 17, 1, 30000), (20, 8, 1, 14000), (20, 28, 1, 800), (15, 29,
    1, 500), (1, 30, 1, 1200);
62
63 -- Reseñas (10)
64
65

```

```

66 INSERT INTO resenas (calificacion, comentario, fecha, id_producto,
    id_cliente) VALUES
67 (5, 'Excelente teléfono, muy rápido.', '2023-10-10', 1, 1),
68 (4, 'Buena laptop, pero la batería dura poco.', '2023-10-11', 11,
    2),
69 (5, 'Me encantó la cámara del S21.', '2023-10-12', 2, 3),
70 (3, 'Los audífonos están bien, un poco caros.', '2023-10-13', 21,
    1),
71 (5, 'Muy potente para programar.', '2023-10-14', 12, 4),
72 (4, 'Buen diseño de audífonos.', '2023-10-15', 22, 3),
73 (5, 'Carga súper rápido.', '2023-10-16', 25, 12),
74 (2, 'La funda se rayó muy rápido.', '2023-10-17', 26, 15),
75 (5, 'El mejor mouse que he tenido.', '2023-10-18', 29, 15),
76 (4, 'Buen teclado, aunque algo ruidoso.', '2023-10-19', 30, 1);

```

4. Consultas y procedimientos almacenados

Para manipular los datos y aplicar las reglas de negocio, se diseñaron consultas y procedimientos almacenados para no depender de la aplicación externa.

4.1. Consultas SQL

Se desarrollaron **SELECT** con **JOIN** y funciones de agregación (**COUNT**, **AVG**).

Código 3: Consulta 1: Listar productos disponibles por categoría, ordenados por precio.

```

1 SELECT p.nombre AS producto, c.nombre AS categoria, p.precio, p.
    stock
2 FROM productos p
3 JOIN categorias c ON p.id_categoria = c.id_categoria
4 WHERE p.stock > 0
5 ORDER BY p.precio ASC;

```

Código 4: Consulta 2: Mostrar clientes con pedidos pendientes y total de compras.

```

1 SELECT c.nombre AS cliente, c.correo, COUNT(pe.id_pedido) AS
    total_pedidos_pendientes
2 FROM clientes c
3 JOIN pedidos pe ON c.id_cliente = pe.id_cliente
4 WHERE pe.estado = 'pendiente'
5 GROUP BY c.id_cliente;

```

Código 5: Consulta 3: Reporte de los 5 productos con mejor calificación promedio en reseñas.

```

1 SELECT p.nombre AS producto, AVG(r.calificacion) AS
    promedio_calificacion
2 FROM productos p
3 JOIN resenas r ON p.id_producto = r.id_producto
4 GROUP BY p.id_producto

```

```

5 ORDER BY promedio_calificacion DESC
6 LIMIT 5;

```

4.2. Procedimientos Almacenados (Store Procedures)

Los procedimientos nos permiten validar condiciones usando IF y frenar transacciones inválidas con SIGNAL.

Código 6: Store Procedures para automatizar reglas de negocio y mantener integridad de datos

```

1  -- Procedimientos almacenados
2
3  -- Cambiamos el delimitador a dos diagonales para que mysql
4  -- entienda que todo el procedimiento es un solo bloque de codigo
5  -- y no se detenga cuando encuentre los puntos y comas internos
6  DELIMITER //
7
8  -- 1. Registrar un nuevo pedido, verificando el límite de 5 pedidos
9  -- pendientes y stock suficiente.
10 -- este procedimiento sirve para automatizar las compras porque
11 -- primero declaramos variables locales para guardar de forma
12 -- temporal los datos que vamos a consultar de las tablas, luego
13 -- usamos SELECT COUNT y SELECT STOCK INTO para meter los
14 -- resultados dentro de nuestras variables y poder hacer las
15 -- validaciones del negocio con un IF, si el cliente ya debe 5
16 -- pedidos o si no hay stock, usamos signal SQLSTATE '45000' para
17 -- detener todo por completo y lanzar un error personalizado,
18 -- evitando que la base de datos se rompa o registre datos falsos;
19 -- si todo esta bien, se inserta el pedido y usamos
20 -- LAST_INSERT_ID para agarrar el id que se acaba de generar en
21 -- automatico y meterlo directo en detalles_pedido
22 CREATE PROCEDURE sp_registrar_pedido(
23   IN p_id_cliente INT,
24   IN p_id_producto INT,
25   IN p_cantidad INT,
26   IN p_precio DECIMAL(10,2)
27 )
28 BEGIN
29   DECLARE v_pendientes INT;
30   DECLARE v_stock INT;
31   DECLARE v_nuevo_id_pedido INT;
32
33   -- contamos cuantos pedidos pendientes tiene este cliente y lo
34   -- guardamos en la variable
35   SELECT COUNT(*) INTO v_pendientes FROM pedidos WHERE id_cliente
36   = p_id_cliente AND estado = 'pendiente';
37
38   -- revisamos cuanto stock tiene el producto actual y lo
39   -- guardamos en la variable
40   SELECT stock INTO v_stock FROM productos WHERE id_producto =
41   p_id_producto;
42
43   IF v_pendientes >= 5 THEN
44     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: El
45     cliente ya tiene 5 pedidos pendientes máximos.';

```



```

27 ELSEIF v_stock < p_cantidad THEN
28     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: No hay
suficiente stock para este producto.';
29 ELSE
30     -- si pasa los filtros guardamos el pedido usando CURDATE
para poner la fecha del dia de hoy en automatico
31     INSERT INTO pedidos (fecha_pedido, estado, id_cliente)
VALUES (CURDATE(), 'pendiente', p_id_cliente);
32
33     -- guardamos el id generado para no perder la relacion con
su detalle
34     SET v_nuevo_id_pedido = LAST_INSERT_ID();
35
36     -- insertamos el producto en la tabla puente de
detalles_pedido
37     INSERT INTO detalles_pedido (id_pedido, id_producto,
cantidad, precio_unitario) VALUES (v_nuevo_id_pedido,
p_id_producto, p_cantidad, p_precio);
38 END IF;
39 END //
40
41
42 -- 2. Registrar una reseña, verificando que el cliente haya
comprado el producto.
43 -- este procedimiento protege la integridad de las opiniones de la
tienda, hacemos un SELECT COUNT cruzando detalles_pedido y
pedidos con un JOIN para verificar si existe al menos un
registro donde este cliente haya pagado por este producto, si
el conteo da cero significa que nunca lo compro, por lo que
usamos SIGNAL SQLSTATE para arrojar un error y bloquear la
insercion de la resena
44 CREATE PROCEDURE sp_registrar_resena(
45     IN p_calificacion INT,
46     IN p_comentario TEXT,
47     IN p_id_producto INT,
48     IN p_id_cliente INT
49 )
50 BEGIN
51     DECLARE v_compro INT;
52
53     -- buscamos si hay registros que demuestren que el cliente si
compro el producto
54     SELECT COUNT(*) INTO v_compro
55     FROM detalles_pedido dp
56     JOIN pedidos p ON dp.id_pedido = p.id_pedido
57     WHERE p.id_cliente = p_id_cliente AND dp.id_producto =
p_id_producto;
58
59     IF v_compro = 0 THEN
60         SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: Solo
puedes hacer reseñas de productos que hayas comprado.';
61     ELSE
62         INSERT INTO resenas (calificacion, comentario, fecha,
id_producto, id_cliente)
63         VALUES (p_calificacion, p_comentario, CURDATE(),
p_id_producto, p_id_cliente);
64     END IF;

```

```

65 END //
66
67
68 -- 3. Actualizar el stock de un producto después de un pedido
69 -- este procedimiento sirve para manteer actualizado el inventario,
    lo que hacemos es un UPDATE directo a la tabla de productos
    usando su llave primaria y le restamos al stock original la
    cantidad exacta de piezas que el cliente se va a llevar en su
    compra
70 CREATE PROCEDURE sp_actualizar_stock(
71     IN p_id_producto INT,
72     IN p_cantidad_comprada INT
73 )
74 BEGIN
75     UPDATE productos
76     SET stock = stock - p_cantidad_comprada
77     WHERE id_producto = p_id_producto;
78 END //
79
80
81 -- 4. Cambiar el estado de un pedido (ej. de pendiente a enviado)
82 -- sirve para la logica de logistica de la tienda, usamos un update
    para modificar el texto del estado de la orden (pendiente,
    enviado, entregado) guiandonos por el id_pedido para no
    alterar las compras de otros clientes
83 CREATE PROCEDURE sp_cambiar_estado_pedido(
84     IN p_id_pedido INT,
85     IN p_nuevo_estado VARCHAR(20)
86 )
87 BEGIN
88     UPDATE pedidos
89     SET estado = p_nuevo_estado
90     WHERE id_pedido = p_id_pedido;
91 END //
92
93
94 -- 5. Eliminar reseña de un producto específico, actualizando el
    promedio de calificaciones
95 -- aqui primero buscamos cual es el id del producto al que
    pertenece la reseña que queremos borrar y lo respaldamos en una
    variable local, ya que si la borramos primero perderiamos ese
    dato; despues de aplicar el delete, hacemos un select combinado
    con AVG para mostrarle de inmediato al administrador como se
    recalculo el promedio general de estrellas de ese producto de
    manera automatica
96 CREATE PROCEDURE sp_eliminar_resena(
97     IN p_id_resena INT
98 )
99 BEGIN
100     DECLARE v_id_producto INT;
101
102     -- respaldamos el id del producto en nuestra variable antes de
    borrar el registro
103     SELECT id_producto INTO v_id_producto FROM resenas WHERE
    id_resena = p_id_resena;
104
105     -- borramos la reseña de la tabla

```

```

106 DELETE FROM resenas WHERE id_resena = p_id_resena;
107
108 -- calculamos y mostramos el nuevo promedio real en pantalla
109 SELECT p.nombre, AVG(r.calificacion) AS nuevo_promedio
110 FROM productos p
111 LEFT JOIN resenas r ON p.id_producto = r.id_producto
112 WHERE p.id_producto = v_id_producto
113 GROUP BY p.id_producto;
114 END //
115
116
117 -- 6. Agregar un nuevo producto, verificando que no exista un
118 -- duplicado (mismo nombre y categoría).
119 -- para evitar errores de captura donde metan dos veces la misma
120 -- laptop o celular, hacemos un select count buscando si ya hay un
121 -- registro con exactamente el mismo nombre dentro de la misma
122 -- categoría, si el sistema encuentra que ya existe, frena la
123 -- operación con un signal y le avisa al usuario
124 CREATE PROCEDURE sp_agregar_producto(
125     IN p_nombre VARCHAR(150),
126     IN p_descripcion TEXT,
127     IN p_precio DECIMAL(10,2),
128     IN p_stock INT,
129     IN p_id_categoria INT
130 )
131 BEGIN
132     DECLARE v_existe INT;
133
134     SELECT COUNT(*) INTO v_existe FROM productos
135     WHERE nombre = p_nombre AND id_categoria = p_id_categoria;
136
137     IF v_existe > 0 THEN
138         SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: Ya
139         existe un producto con ese nombre en esta categoría.';
140     ELSE
141         INSERT INTO productos (nombre, descripcion, precio, stock,
142         id_categoria)
143         VALUES (p_nombre, p_descripcion, p_precio, p_stock,
144         p_id_categoria);
145     END IF;
146 END //
147
148
149 -- 7. Actualizar la información de un cliente (ej. dirección o telé
150 -- fono)
151 -- este procedimiento sirve para que el usuario pueda cambiar sus
152 -- datos de contacto en su perfil, aplicamos un UPDATE directo
153 -- sobre la tabla clientes y usamos el id_cliente en el WHERE para
154 -- que se modifiquen los datos de la persona correcta
155 CREATE PROCEDURE sp_actualizar_cliente(
156     IN p_id_cliente INT,
157     IN p_nueva_direccion VARCHAR(255),
158     IN p_nuevo_telefono VARCHAR(20)
159 )
160 BEGIN
161     UPDATE clientes
162     SET direccion = p_nueva_direccion, telefono = p_nuevo_telefono

```

```

151     WHERE id_cliente = p_id_cliente;
152 END //
153
154
155 -- 8. Generar un reporte de productos con stock bajo (menos de 5
      unidades)
156 -- es una consulta de control para el administrador de la tienda,
      hace un SELECT filtrando con un WHERE para extraer unicamente
      los productos que tengan menos de 5 piezas en el inventario,
      ordenandolos de menor a mayor stock para saber que urge
      resurtir primero.
157 CREATE PROCEDURE sp_reporte_stock_bajo()
158 BEGIN
159     SELECT nombre, stock, precio
160     FROM productos
161     WHERE stock < 5
162     ORDER BY stock ASC;
163 END //
164
165 -- regresamos el delimitador al punto y coma que se usa de forma
      normal en sql
166 DELIMITER ;

```